

Inigo Quilez :: articles :: xor SDFs - 2023

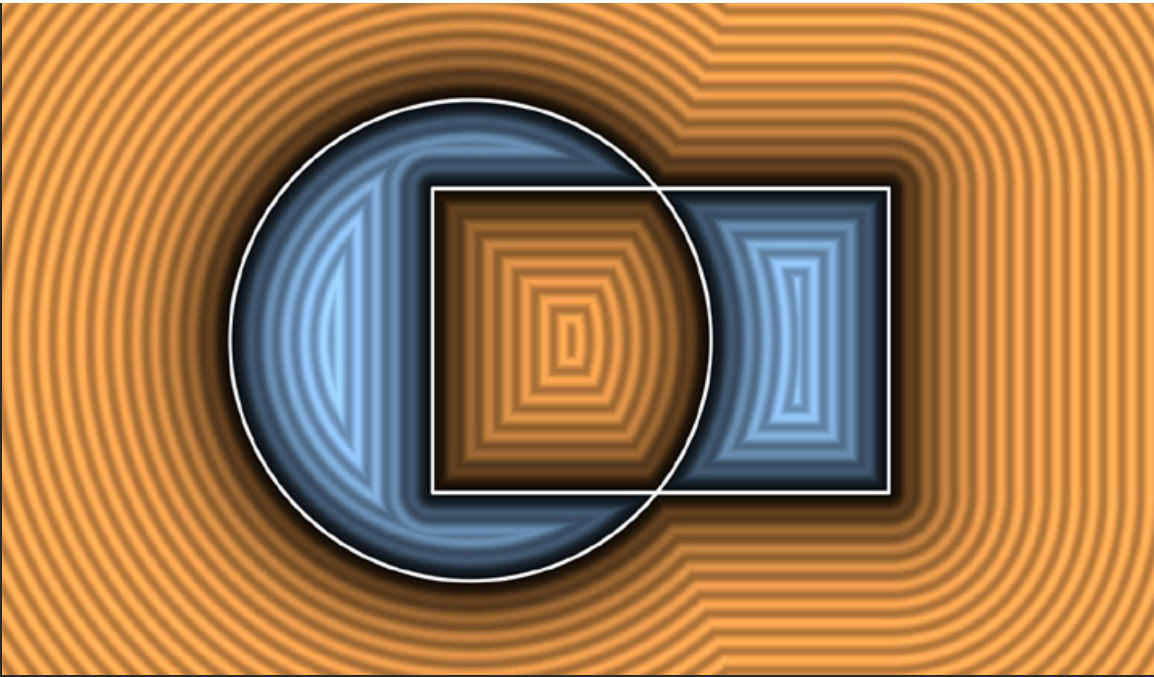


Intro

It is well known that the **union operator** of SDFs implemented with `min()` is not a correct SDF in the interior of the resulting shape. That doesn't mean we can't use it, on the contrary, many algorithms can work with it because all they require is the exterior distance to be correct. We just wished it was correct in the interior too. It's also well known that the **subtraction and intersection operators** implemented with `max()` also produce incorrect SDFs.

Now, given two SDF objects **a** and **b** to be combined into a new shape **c**, we can try to find other operators beyond union, subtraction and intersection, if any, and study how those behave. Actually, are there other operators? Well, we know that with union, subtraction and intersection, the basic three, we can produce four different outcomes. It's four and not three because subtraction comes in two variants, depending on which shape you subtract from the other. But from a constructive perspective, we can easily arrive at the conclusion that it should be possible to combine two shapes in at least eight different ways; not just four... Let's convince ourselves of that by looking at the overlaps generated by the two shapes. These belong in one of three classes - regions of **a** not in **b**, regions of **b** not in **a**, and regions from the overlap of **a** and **b**. Now, each one of these three classes can be in the interior or the exterior of the new shape **c**. So that means there are eight (2^3) possible resulting shapes that we can construct, and so eight different operations or ways to combine two shapes! But we already know four of them that come from union, subtraction and intersection... so what are the other missing four?

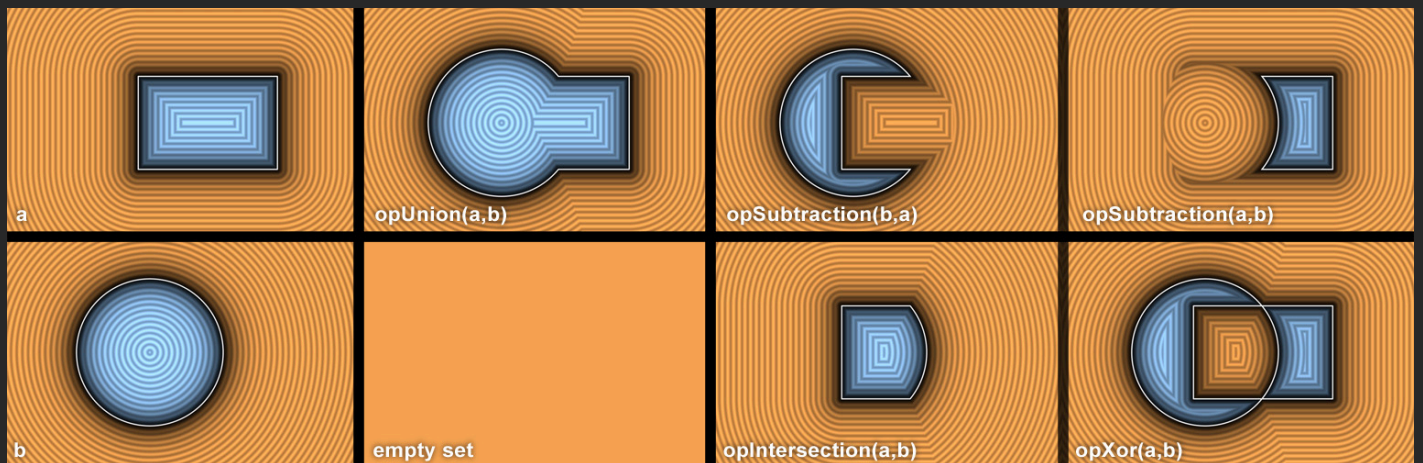
Well, three of those are pretty boring: just **a**, just **b** or the empty set. But there is one last operation that is both uncommon yet still interesting: the xor of **a** and **b**, which includes all parts of **a** and **b** that do not overlap with each other, but excludes those that are common:



xor SDF

Now, it is not trivial to find cases where the xor would be a useful operation in the context of 3D solid modeling. And maybe there isn't any; but that doesn't stop the very amazing computer graphics explorers that you find hanging around at Shadertoy from playing and poking at the problem.

And so, Shadertoy user "[jt](#)", a.k.a. Jakob Thomsen, implemented an **XOR operator** for SDFs, and was quickly surprised to note that, unlike the popular union, subtraction and intersection operators, he couldn't make this one fail and produce a non distance field, no matter how much one tries. The question then was, was this just an accident due to the simplicity of the specific shapes **a** and **b** chosen, or was this a true and general property of the xor operator for SDFs?



all 8 possible combinations of two SDFs

As with many other little gems discovered all across Shadertoy, collaboration was the name of the game, and I was able to have a look to the problem and provide a proof that indeed, for two SDF shapes **a** and **b**, the xor operation returns always, unconditionally, a correct SDF. Here it's the proof:

The proof

So, the xor operator jt implemented was this:

```
float opXor( float a, float b )
{
    return max( min(a,b), -max(a,b) );
}
```

This is just one of the many possible implementations of xor; because we can always use what I'll call the "min-max substitution",

$$\min(a,b) = -\max(-a,-b)$$

and with it, construct the eight different ways of rewriting the expression:

```
xor(a,b) = max( -max(-a,-b), min(-a, -b));
xor(a,b) = max( min( a, b), -max( a, b));
xor(a,b) = max( min( a, b), min(-a, -b));
xor(a,b) = max( -max(-a,-b), -max( a, b));
xor(a,b) = -min( max(-a,-b), -min(-a, -b));
xor(a,b) = -min( -min( a, b), max( a, b));
xor(a,b) = -min( -min( a, b), -min(-a, -b));
xor(a,b) = -min( max(-a,-b), max( a, b));
```

It doesn't matter which one we pick as a starting point for our proof, so I'll just stick to jt's. But do remember the min-max substitution, because we'll use it shortly.

Besides the min-max substitution, the second weapon at our disposal for proving the correctness of xor is the fact that the min() of two SDFs, the union, is correct IF the two input SDFs are positive. See again [the article on exterior distances](#) if you need a reminder.

So it just makes sense that we study the behavior of xor based on the signs of the two input SDFs **a** and **b**,

and that we consider the four possible cases separately:

Case 1 : $a > 0, b > 0$

Case 2 : $a < 0, b > 0$

Case 3 : $a > 0, b < 0$

Case 4 : $a < 0, b < 0$

Okey, let's start:

Case 1: We have $\text{xor}(a,b) = \max(\min(a,b), -\max(a,b))$ with $a > 0$ and $b > 0$. But because both a and b are positive, then both $\min(a,b)$ and $\max(a,b)$ must be too. So the first argument to the outer max is positive (from the min) and the second one is negative (from the -max). So the first one will always win, meaning the whole expression simplifies to $\text{xor}(a,b) = \min(a,b)$

Case 2: We have $\text{xor}(a,b) = \max(\min(a,b), -\max(a,b))$ with $a < 0$ and $b > 0$. So the inner $\min(a,b)$ reduces simply to a , and the inner max to simply b . So $\text{xor}(a,b) = \max(a, -b)$. But by the min-max substitution, $\text{xor}(a,b) = -\min(-a, b)$.

Case 3: Now $\text{xor}(a,b) = \max(\min(a,b), -\max(a,b))$ with $a > 0, b < 0$. So now the inner min reduces to b , and the inner max reduces to a . So we have $\text{xor}(a,b) = \max(b, -a)$. Again by the min-max substitution, we have $\text{xor}(a,b) = -\min(a, -b)$.

Case 4: Lastly, now $\text{xor}(a,b) = \max(\min(a,b), -\max(a,b))$ with $a < 0, b < 0$. So the inner min and max are both negative, which means the inner -max will always win over the min during the evaluation of the outer max. So, $\text{xor}(a,b) = -\max(a,b)$. But by the min-max substitution, $\text{xor}(a,b) = \min(-a, -b)$.

In summary, we have:

Case 1 : $a > 0, b > 0$: $\text{xor}(a,b) = \min(a, b)$

Case 2 : $a < 0, b > 0$: $\text{xor}(a,b) = -\min(-a, b)$

Case 3 : $a > 0, b < 0$: $\text{xor}(a,b) = -\min(a, -b)$

Case 4 : $a < 0, b < 0$: $\text{xor}(a,b) = \min(-a, -b)$

Now note that in all four cases, both arguments to the min function are positive, because the minus signs cancel perfectly with the signs of the arguments for each one of the four cases. This means that the $\min()$ in all these four expressions is like the union of two exterior distances, and so it always produces a correct SDF. After computing that distance, the negative signs from of the min for cases 2 and 3 mark the field as an interior

distance.

So in all four cases the xor function is returning the correct distance, and so our proof that the xor of two SDFs is also an SDF is complete.

Now, there's an observation that we could have done right from the beginning that could have lead us to this same result right away: the xor of **a** and **b** shares the same isosurface (zero level set, ie, the white border in the first picture) with both **a** and **b**. So as developer "Boris The Brave" noted in social media, it must always be expressible as the min of **a** and **b**, corrected with the right signs depending on the four cases listed above. So there, a much shorter path to the same result.

inigo quilez - learning computer graphics since 1994